

Exercice 1

1. CHERCHEZ L'ERREUR ! Ce programme est bogué :

```
0: #include <iostream>
1: int main()
2:{
3: cout << "Bonjour !" << endl;
4: return 0;
5: }
```

2. Proposez trois solutions pour résoudre le problème du programme précédent.

3. Écrivez le code permettant de déclarer un espace de noms **MesElements**. Cet espace de noms doit contenir une classe appelée **MaClasse**.

Exercice 2

Créer une classe **point** ne contenant qu'un constructeur sans arguments, un destructeur et un membre donnée privé représentant un numéro de point (le premier créé portera le numéro 1, le suivant le numéro 2...). Le constructeur affichera le numéro du point créé et le destructeur affichera le numéro du point détruit.

Écrire un petit programme d'utilisation créant dynamiquement un tableau de **4 points** et le détruisant.

Exercice 3

1. Écrivez la déclaration de la classe **Personnel** contenant les données membres **sonAge**, **sonAnciennete** et **sonSalaire**.

2. Modifiez la déclaration de la classe **Personnel** de sorte que ses données membres soient privées et que des méthodes d'accès puissent lire et mettre à jour chacune d'elles.

3. Écrivez un programme qui crée deux objets de la classe **Personnel**. Pour chacun, définissez l'âge, l'ancienneté et le salaire, puis affichez ces valeurs. Vous devrez aussi ajouter le code des méthodes d'accès.

4. Ecrivez le code d'une méthode qui affiche le salaire de chaque employé, arrondi à la centaine de dirhams la plus proche.

5. Modifiez de nouveau la classe **Personnel** de façon à initialiser les données membres **sonAge**, **sonAnciennete** et **sonSalaire** lors de la création d'un objet.

Exercice 4

Ecrire un programme contenant la classe **Personne** (**Nom**, **Prenom**, **age**) et qui demande à l'utilisateur combien de personnes il veut encoder. Puis l'utilisateur encode les noms, prénoms et âges des personnes.

La classe **Personne** doit contenir une méthode publique qui permettant à une personne de se présenter.

Le programme principal va trier par âge (tri à bulles) les personnes en utilisant une surcharge d'opérateur de comparaison et va ensuite demander aux personnes de se présenter par ordre d'âge.

Exercice 5

1. Créer un patron de fonctions permettant de calculer la somme d'un tableau d'éléments de type quelconque, le nombre d'éléments du tableau étant fourni en paramètre (on supposera que l'environnement utilisé accepte les « paramètres expression »). Écrire un petit programme utilisant ce patron.
2. Créer un patron de classes nommé **pointcol**, tel que chaque classe instanciée permette de manipuler des points colorés (deux coordonnées et une couleur) pour lesquels on puisse « choisir » à la fois le type des coordonnées et celui de la couleur. On se limitera à deux fonctions membre : un constructeur possédant trois arguments (sans valeur par défaut) et une fonction affiche affichant les coordonnées et la couleur d'un « point coloré ».

Exercice 6

Faire un jeu au vous demandez aux deux joueurs d'encoder un nom de personnage ainsi que de choisir une classe de **personnage**. Les joueurs ont le choix parmi 3 classes (**Magicien, Sniper et Barbare**). Ces classes héritent de la classe **personnage**.

La classe **personnage** est une classe abstraite contenant les méthodes publiques :

void recevoir_degats(int degat) ;

void frapper(Personnage *cible) ;

void bouclier() ; (permet au joueur de mieux encaisser la prochaine attaque) et une méthode virtuelle pure :

attaque_speciale(Personnage *cible) ;

Les classes filles doivent redéfinir ces méthodes à leur façon.

Votre programme va ensuite demander en boucle aux joueurs l'action à effectuer (frapper, bouclier ou attaque spéciale) jusqu'à qu'il n'y ait plus qu'un seul survivant.

Exercice (Facultatif)

Soit une classe **vecteur3d** définie comme suit:

```
class vecteur3d{
float x,y,z;
public:
vecteur3d(float c1=0.0, float c2=0.0, float c3=0.0)
{x=c1; y=c2; z=c3;}
....
};
```

1. Introduire une fonction membre nommée **coincide** permettant de savoir si deux vecteurs ont les mêmes composantes :

- (a) en utilisant une transmission par valeur ;
- (b) en utilisant une transmission par adresse ;
- (c) en utilisant une transmission par référence.

Si **v1** et **v2** désignent 2 vecteurs de type **vecteur3d**, comment s'écrit le test de coïncidence de ces 2 vecteurs, dans chacun des 3 cas considérés ?

2. Introduire une fonction membre nommée **normax** permettant d'obtenir parmi deux vecteurs, celui qui a la plus grande norme. On prévoira trois situations :

- (a) le résultat est renvoyé par valeur ;
- (b) le résultat est renvoyé par référence, l'argument étant également transmis par référence ;
- (c) le résultat est renvoyé par adresse, l'argument étant également transmis par adresse.

3. Définir le constructeur de recopie correspondant.

4. Surcharger l'opérateur + afin que les instructions suivantes seront acceptées par le compilateur :

vecteur3d v1(2,3,4), v2(0,1,-1), v3;

v3 = v1+v2; // v3 sera(2,4,3)

5. On souhaite modifier les composantes d'un vecteur à partir de la fonction **cin (cin>>v1)**. Donner la fonction surchargée correspondante.

6. Surcharger la fonction cout (**cout<<v2**).

7. Adapter la classe **vecteur3d** pour qu'elle dispose d'une fonction membre nombre fournissant le nombre de points existant à un instant donnée.

8. Donner un programme (**main**) permettant d'appeler les différentes fonctions membre de la classe **vecteur3d**.